Seeded adversarial layouts: first verified experiment

HyMeKo research workspace

Created 2026-09-29 02:36 CEST

Scope and ETA. A bounded research client generates grid layouts, compiles them to the existing canonical IR, measures existing Graph searches, and exports selected scenes through the existing Gazebo template pipeline. All simulation and campaign execution occurs on Amaterasu. Estimated completion: 60–90 minutes, based on implementation, focused tests, remote build, a 1000-candidate experiment and artifact review. This is an estimate, not a deadline promise.

Ownership. Core owns compilation, IDs, incidence indexing and semantic hashes. Graph owns BFS, A* and their work counters. Emitter owns canonical HyMeKo source output. Formats owns SDF generation. The example is experiment composition, not a new production world model or graph engine. The simulator's dirty checkout is not modified. The separate metaheuristics pilot has a different worker and files.

Files. This plan's four files and:

- `hymeko_formats/Cargo.toml`
- `hymeko_formats/examples/adversarial_grid.rs`
- `transforms/gazebo/queries.hymeko`
- `transforms/gazebo/template.world.sdf`
- `hymeko_formats/tests/gazebo_obstacles.rs`
- `hymeko_formats/tools/adversarial_grid_artifacts.py`
- `hymeko_formats/tools/run_adversarial_grid.sh`
- `hymeko_formats/tools/validate_adversarial_grid_gazebo.py`
- `hymeko_emitter/src/emit_hymeko.rs`: use the existing Core `build_path_index` API; the pinned revision lacks the constructor currently called.
- `hymeko_query/tests/test_prop_witnesses.rs`: import the pinned Core semantic digest under its existing versioned name so the integrated tests compile.
- `reports/2026-09-29-adversarial-grid.md`

Generated artifacts live below that report's companion directory. Cargo.lock changes only if dev-dependency bookkeeping requires it. No CORE.YAML item is modified and no public Rust API changes.

Interfaces. The example accepts output directory, seed count, candidates per seed and bounded grid dimensions. Defaults: 16 by 16, 10 seeds, 100 proposals per seed. Exported obstacle nodes inherit `gazebo_static_box` and carry numeric x, y, z, size_x, size_y, size_z attributes. The template adds static box models with collision and visual geometry; scenes without such nodes retain their existing semantics. Existing Graph and Emitter crates are reused as dev dependencies.

Method. Keep start, goal, grid dimensions and wall count fixed. Use seeded proposals; reject unreachable worlds. Baseline is the first feasible proposal; the selected world maximizes existing Manhattan A* expanded-node count among the same bounded proposal set. This is best-of-budget selection, not a universal hardness proof. Existing BFS supplies a path witness; zero-heuristic search is a secondary measurement. Save every proposal outcome, generation cost, configuration, source and solver identity. Report selection cost separately from solving cost.

Validation. Before the campaign: seed replay/variation, known-maze optimal path, endpoint and obstacle validity, canonical source round trip and semantic hash, SDF count/position/size checks. Run existing Gazebo template tests. Audit `cargo tree -d`; Core/Graph identities must agree. On Amaterasu run the campaign, validate SDF with installed Gazebo tooling, and run a finite headless Gazebo simulation in its own transport partition. Check simulation time advanced, expected obstacle count and poses, and no model-loading errors. Do not mistake file parse success or process startup for a completed simulation.

Budget and risk. Grid dimensions at most 16, at most 1000 proposals for the first campaign, five-minute campaign cap, solver memory target below 2 GiB; remote compilation uses at most two jobs and is logged separately. Gazebo uses a finite iteration count and 60-second timeout. No paid model calls or GPU training. Long paths must not hit the legacy u8 BFS distance cap. Source metadata must survive round trip; EditorIR bridge metadata loss is why source-first compilation is used. Disconnected proposals are rejected rather than scored as maximally hard. Tie breaking and failure outcomes are recorded. Existing Amaterasu services are untouched.

Rollback and delivery. Remove only task-added files and review the named template/manifest diffs to revert them; preserve other workers' changes. No commit, push or branch switching. Remote source copies are explicitly labeled build snapshots of the local revision plus changes, not clean-clone release evidence. Deliver JSON, plot, layout images and a path GIF; archive the report with the existing report-sync tool. Record exact remote path, commands, logs and outcomes.

